Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение высшего образования «Омский государственный технический университет»
Кафедра «Автоматизированные системы обработки информации и управления»

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

по дисциплине «Кроссплатформенная разработка»
студента Колохматова Андрея Максимовича группы ПИН-252т

Пояснительная записка
Шифр работы От-43-ПИН-252т-2 ЛР
Специальность 09.03.03 Программная инженерия

Преподаватель М.А. Мищенко
Студент А.М. Колохматов

Омск 2026

РЕФЕРАТ

Объектом выполненной работы является кроссплатформенное приложение «Рэп-каталог», разработанное на Flutter и языке Dart.

Целью данной работы является построение каркаса приложения на Flutter: навигации между экранами, экрана-списка и экрана детали на моковых данных, поиска, переключения темы и локализации интерфейса.

В работе применены структура кода по фичам со слоями data, domain и presentation, управление состоянием на flutter_bloc (Cubit) с закрытыми неизменяемыми состояниями, навигация go_router, механизм локализации gen-l10n на ARB-файлах и тема Material 3.

Степень внедрения — для учебных целей.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1 ПОСТАНОВКА ЗАДАЧИ 6
2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ 7
3 РЕАЛИЗАЦИЯ 8
3.1 Модель предметной области и репозиторий 8
3.2 Моковые данные 8
3.3 Состояния и Cubit 8
3.4 Навигация 9
3.5 Тема 9
3.6 Локализация 9
3.7 Адаптивность 10
4 ПРОВЕРКА РАБОТЫ 11
ЗАКЛЮЧЕНИЕ 16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 17

ВВЕДЕНИЕ

Flutter — фреймворк для построения интерфейсов на языке Dart, который рисует интерфейс собственным движком, а не элементами платформы. Благодаря этому один и тот же код выглядит и ведёт себя одинаково в браузере, на настольных системах и на мобильных устройствах, а платформенные проекты служат только точками входа.

Предметная область та же, что в первой лабораторной работе на Compose Multiplatform, — каталог хип-хоп исполнителей. Источник данных курса — открытый программный интерфейс MusicBrainz, но на этой вехе сеть не используется: приложение работает на моковых данных, выгруженных из MusicBrainz, и повторяет их форму, поэтому на следующей вехе моки заменяются сетью без изменения экранов.

Цель работы — перенести каркас приложения на Flutter так, чтобы он не рассыпался при обращении с ним: при переключении темы и языка, переходе на деталь и возврате назад, при поиске и при изменении размера окна.

Для достижения цели поставлены следующие задачи:

— создать проект Flutter из мастера с таргетами веб и настольная система;

— разложить код по фичам: features/artist с папками data, domain и presentation, общий код — в common;

— перенести модель предметной области и моковый набор из 33 артистов из первой работы;

— реализовать экран-список в виде сетки карточек и экран детали со справкой, дискографией, тегами и переходами к связанным исполнителям;

— добавить поиск по имени исполнителя: подстрока сужает сетку, запрос сохраняется при возврате с детали, при отсутствии совпадений выводится «Ничего не найдено»;

— выбрать и обосновать способ управления состоянием, описать состояния экранов закрытыми неизменяемыми типами;

— организовать навигацию на go_router, где в пути передаётся идентификатор, а не модель;

— вынести цвета в тему и реализовать переключение светлого и тёмного оформления;

— вынести подписи интерфейса в ARB-файлы на русском и английском языках и проверять совпадение ключей сценарием;

— проверить работу приложения в браузере, в том числе в узком окне.

1 ПОСТАНОВКА ЗАДАЧИ

Приложение состоит из двух экранов: списка артистов в виде сетки и детали артиста. Карточка списка показывает фотографию, имя и ведущий жанр. Деталь содержит сведения, которых нет в списке: уточнение из MusicBrainz, жанры, страну и место рождения, годы жизни или существования группы, короткую справку, псевдонимы, альбомы, теги с весом и связанных артистов, по которым можно перейти дальше. Над сеткой находится поле поиска по имени исполнителя.

Требования к каркасу приведены в таблице 1.

Таблица 1 — Требования к каркасу

Область	Требование
Структура	Код разложен по фичам, у фичи слои data, domain и presentation; репозиторий описан интерфейсом и регистрируется в корне приложения
Состояние	Выбран и обоснован один способ управления состоянием; состояние экрана — один неизменяемый тип без противоречивых флагов; ответ после закрытия экрана не вызывает ошибку
Навигация	В пути передаётся идентификатор, а не модель; «назад» со связанного артиста возвращает на предыдущую деталь; адрес в браузере меняется
Оформление	Цвета и картинки не записаны в виджетах; светлая и тёмная тема переключаются
Локализация	Подписи в ARB-файлах на двух языках, ключи и параметры совпадают
Адаптивность	Сетка перестраивается в узком окне, на широком экране ширина содержимого ограничена

2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ

Код разложен по фичам. Единственная фича каталога — features/artist — разделена на три слоя. Слой domain содержит модель ArtistModel с вложенными моделями и интерфейс репозитория IArtistRepository. Слой data содержит моковые данные и реализацию репозитория ArtistRepository. Слой presentation содержит Cubit и состояния в папке bloc, экраны и их виджеты в папке view и расширения для показа данных в папке utils. Общий код лежит в common: навигация, настройки темы и языка, тема и общие виджеты.

Зависимости направлены к слою domain: presentation знает только интерфейс репозитория и модели, data реализует интерфейс. На следующей вехе моковая реализация заменяется сетевой, и правка ограничивается папкой data и сборкой зависимостей в корне.

Корневой виджет App собирает зависимости: RepositoryProvider регистрирует ArtistRepository по интерфейсу IArtistRepository, MultiBlocProvider создаёт SettingsCubit и ArtistListCubit, а MaterialApp.router получает тему, язык и роутер. Cubit списка живёт в корне, поэтому запрос поиска и найденные записи переживают переход на деталь и возврат. Cubit детали создаёт сам экран детали через BlocProvider, и он закрывается вместе с экраном.

Для управления состоянием выбран flutter_bloc в варианте Cubit. Действия на экранах прямые — «загрузи», «найди», «загрузи артиста по идентификатору», — поэтому публичные методы Cubit проще событий Bloc и не теряют в проверяемости: состояние меняется только внутри Cubit, а экран лишь рисует его и вызывает методы. Навигацию выполняет виджет через context.push, Cubit о ней не знает.

3 РЕАЛИЗАЦИЯ

3.1 Модель предметной области и репозиторий

Модель ArtistModel повторяет форму ответа MusicBrainz: идентификатор — строка MBID, есть имя, имя для сортировки, уточнение, тип (исполнитель, группа, другое), страна, место рождения, даты, жанры, теги, псевдонимы, релиз-группы, связанные артисты и справка на двух языках. Жанры описаны перечислением Genre из 25 значений. Модели неизменяемые и наследуют Equatable, артисты сравниваются по идентификатору.

Интерфейс IArtistRepository содержит методы getArtists(query), getArtist(id) и getRelated(artist). Все они возвращают Future, хотя моки отвечают сразу: так экраны уже сейчас написаны под асинхронный источник, и переход на сеть не меняет их код.

3.2 Моковые данные

Моки содержат 33 записи, выгруженные из MusicBrainz, — те же, что в первой работе: имена, страны, даты, теги и 410 релиз-групп являются настоящими. Справка — первый абзац статей английской и русской Википедии. Фотографии артистов хранятся в ресурсах приложения, взяты с Викисклада, авторы и лицензии указаны в docs/CREDITS.md. Класс ImageSources сопоставляет MBID и файл снимка; если снимка нет, рисуется плитка с первой буквой имени на градиенте цветов жанров.

3.3 Состояния и Cubit

Состояние каждого экрана — один закрытый (sealed) тип. Список: Loading | Loaded(artists, query) | NotFound(query). Деталь: Loading | Loaded(artist, related) | NotFound. Противоречивых сочетаний флагов вида «загружается и уже не найдено» не бывает, а выражение switch в виджете обязано перечислить все варианты, иначе код не скомпилируется.

ArtistListCubit запоминает последний запрос поиска и отбрасывает ответы на предыдущие, чтобы устаревший результат не перетёр свежий. После каждого await в Cubit проверяется isClosed: если экран закрыт, пока шла загрузка, новое состояние не отправляется.

Поле поиска хранит текст в TextEditingController, который создаётся в initState и освобождается в dispose. Поскольку Cubit списка живёт в корне, при создании экрана поле восстанавливает текст из его состояния. Крестик в поле очищает и текст, и запрос в Cubit. Сравнение идёт без учёта регистра, пробелы по краям отбрасываются.

3.4 Навигация

Навигация построена на go_router. Маршрут «/» — список, вложенный маршрут «artist/:id» — деталь; в пути передаётся MBID, а не модель, поэтому ссылку на деталь можно открыть напрямую, а «назад» с неё ведёт в список благодаря вложенности маршрутов. Связанный артист открывается через push, а не go, поэтому «назад» возвращает на предыдущую деталь, а не в список.

Роутер создаётся один раз как статическое поле: при смене темы и языка MaterialApp перестраивается, но стек экранов не теряется. Параметр GoRouter.optionURLReflectsImperativeAPIs включён, чтобы push менял адрес в браузере. Сетке задан PageStorageKey, и позиция прокрутки сохраняется, пока поверх открыта деталь.

3.5 Тема

AppTheme строит светлую и тёмную темы Material 3 из ColorScheme. Цвета, которых нет в схеме, собраны в AppColors, в том числе цвета жанров: они несут смысл данных и не меняются вместе с темой. В виджетах нет ни Colors.*, ни строковых литералов.

Тема и язык хранятся в SettingsCubit одним неизменяемым состоянием с методом copyWith. Переключатели расположены в шапке приложения. Настройки живут в памяти сеанса: сохранение на диск относится к следующей вехе курса.

3.6 Локализация

Подписи интерфейса вынесены в ARB-файлы lib/l10n/app_ru.arb (шаблон) и app_en.arb. По ним механизм gen-l10n генерирует класс AppLocalizations, и в коде стоят сгенерированные методы, а не строковые ключи: опечатка не компилируется. Счётчики и подписи с подстановками описаны параметрами.

Совпадение ключей и параметров в двух файлах проверяется сценарием tool/check_arb.dart: ключ, забытый в одном языке, при переключении оставил бы на экране чужой язык. Локализуется интерфейс, а не данные каталога. Исключение — справка об исполнителе: у неё есть отдельная русская версия из русской Википедии, а не перевод.

3.7 Адаптивность

Сетка выбирает число колонок по доступной ширине через LayoutBuilder: три колонки от 720 логических пикселей, две — от 300, иначе одна. Виджет ContentFrame ограничивает ширину содержимого 1040 пикселями, чтобы на широком мониторе карточки и текст не растягивались.

4 ПРОВЕРКА РАБОТЫ

Статический анализ flutter analyze замечаний не выдаёт. Сценарий tool/check_arb.dart подтверждает совпадение 50 ключей в двух ARB-файлах.

Работа приложения проверялась в браузере Google Chrome: прокрутка списка, переход на деталь и возврат, переход к связанному артисту, переключение темы и языка, поиск по подстроке имени, возврат с детали с сохранением запроса, поиск несуществующего имени и узкое окно.

Внешний вид приложения приведён на рисунках 1–5, работа поиска — на рисунках 6–8, узкое окно — на рисунке 9.

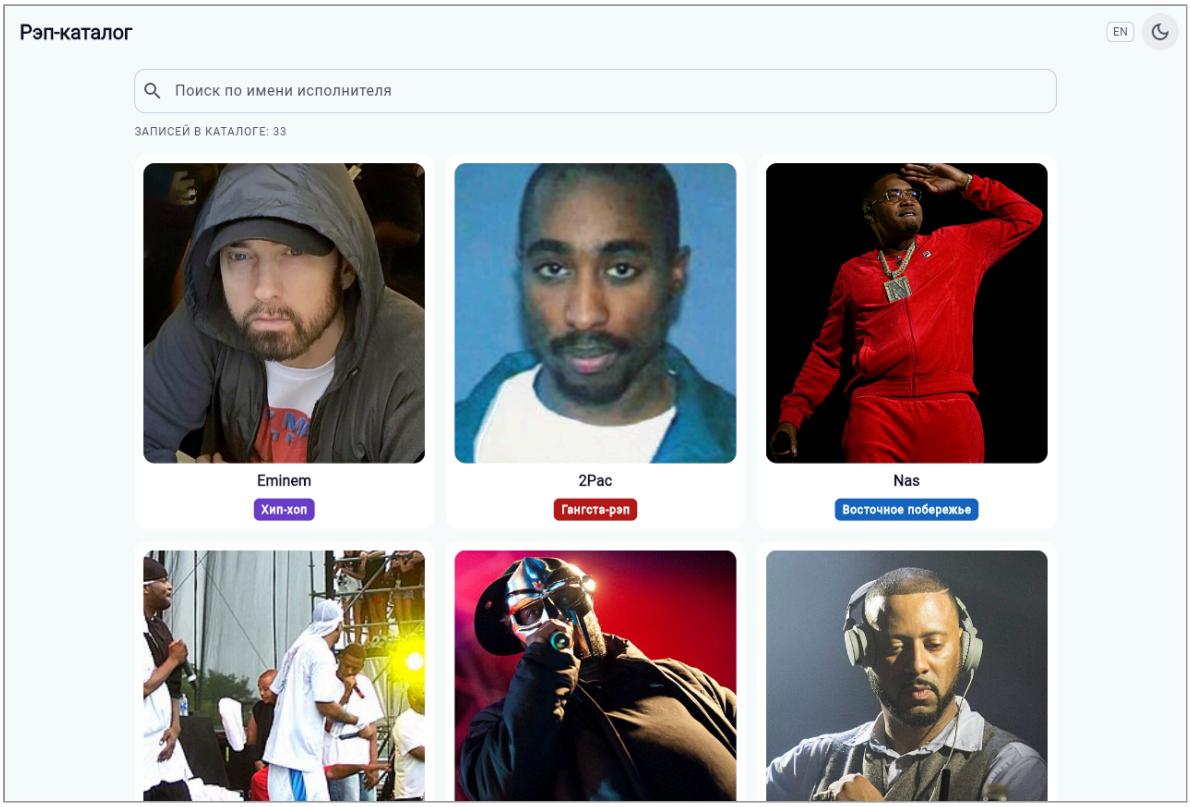


Рисунок 1 — Экран списка артистов, светлая тема

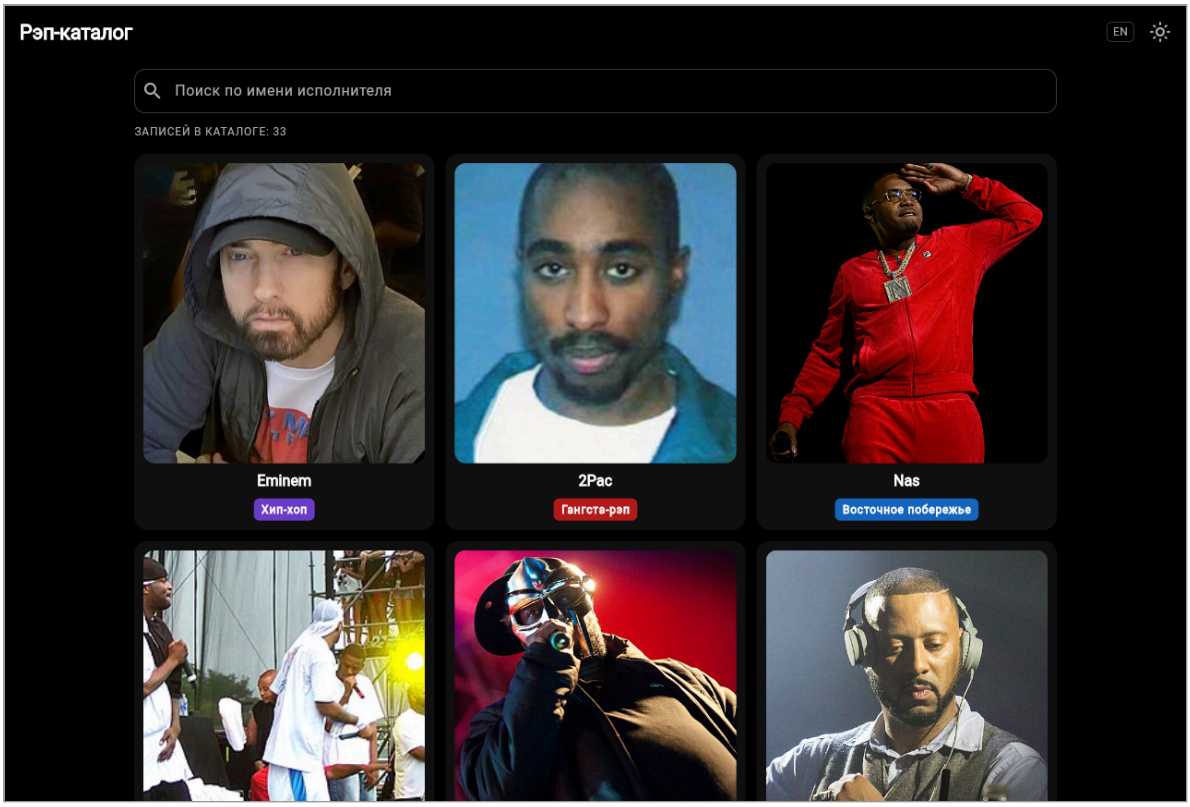


Рисунок 2 — Экран списка артистов, тёмная тема

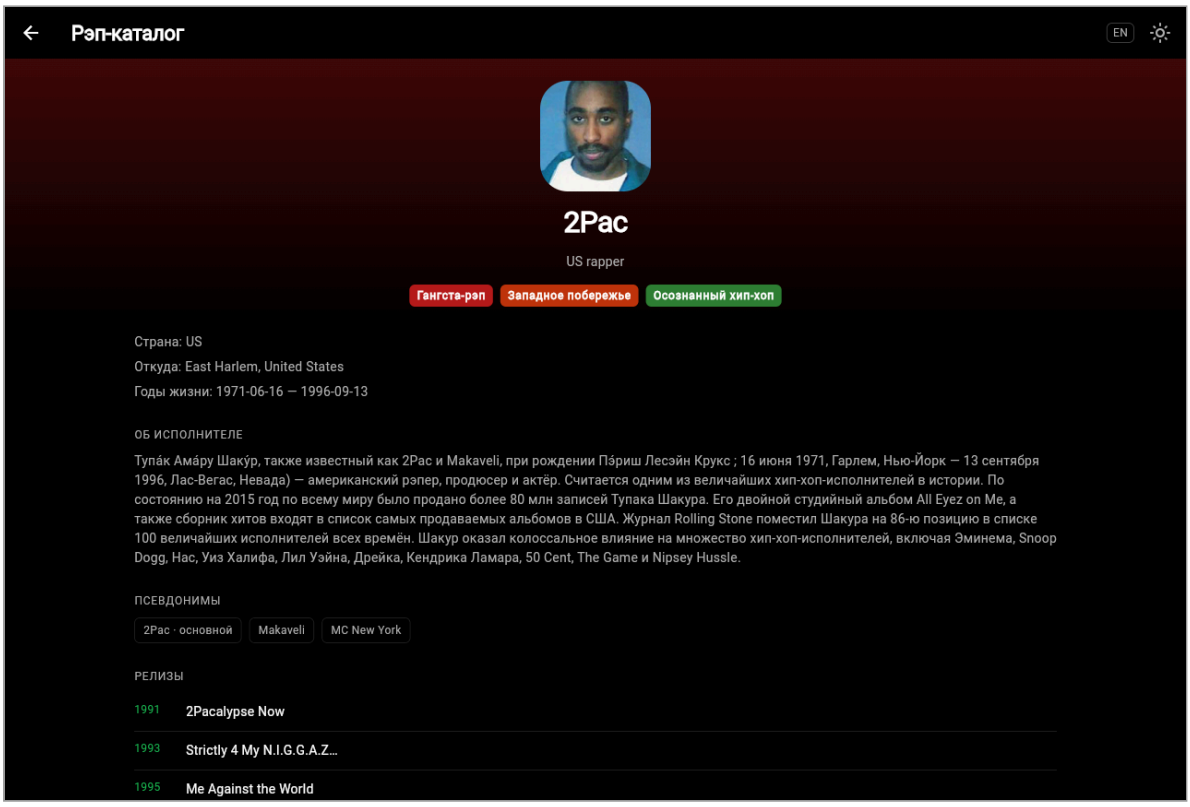


Рисунок 3 — Экран детали артиста, тёмная тема

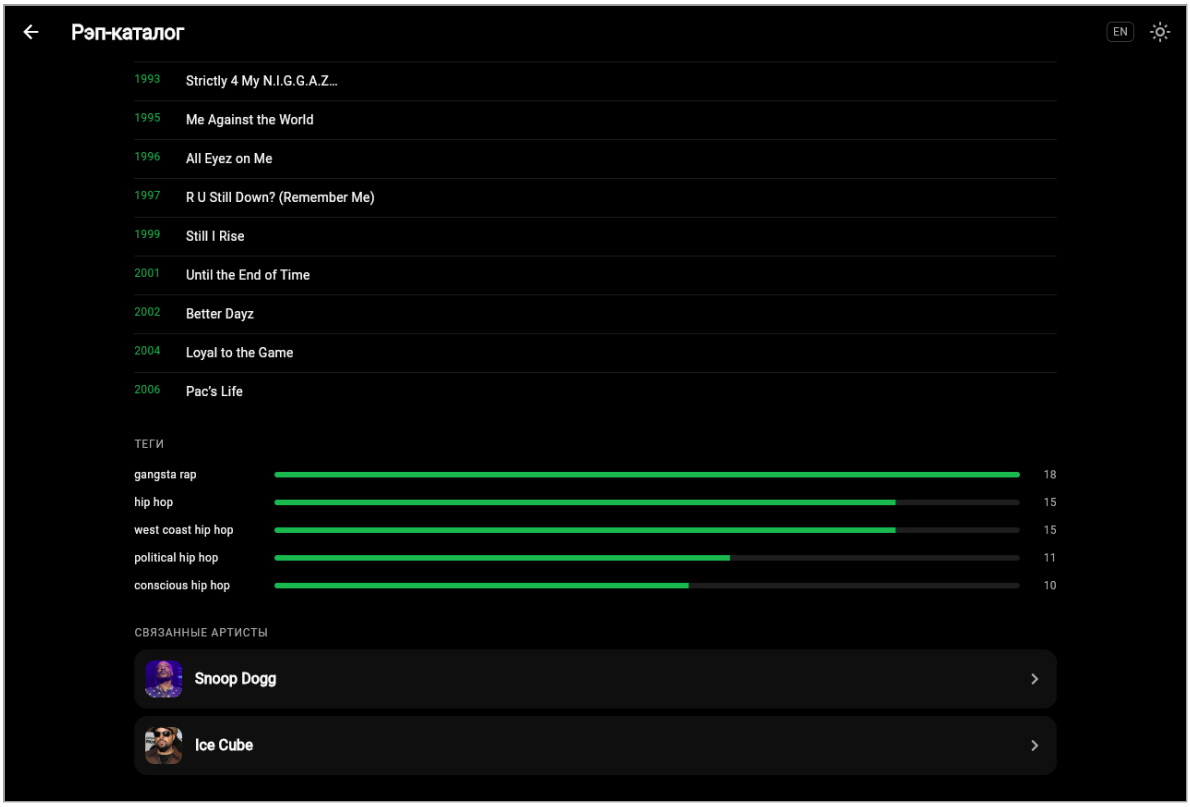


Рисунок 4 — Деталь артиста: альбомы, теги и связанные артисты

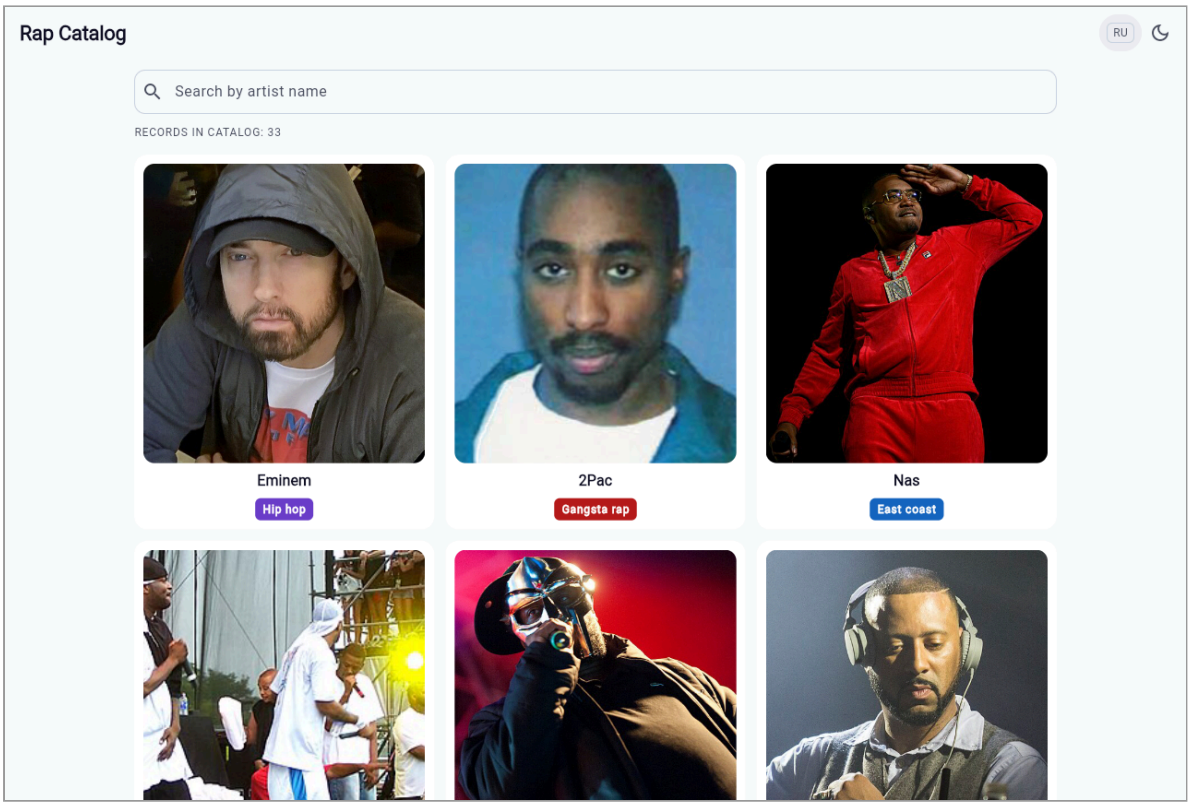


Рисунок 5 — Интерфейс на английском языке

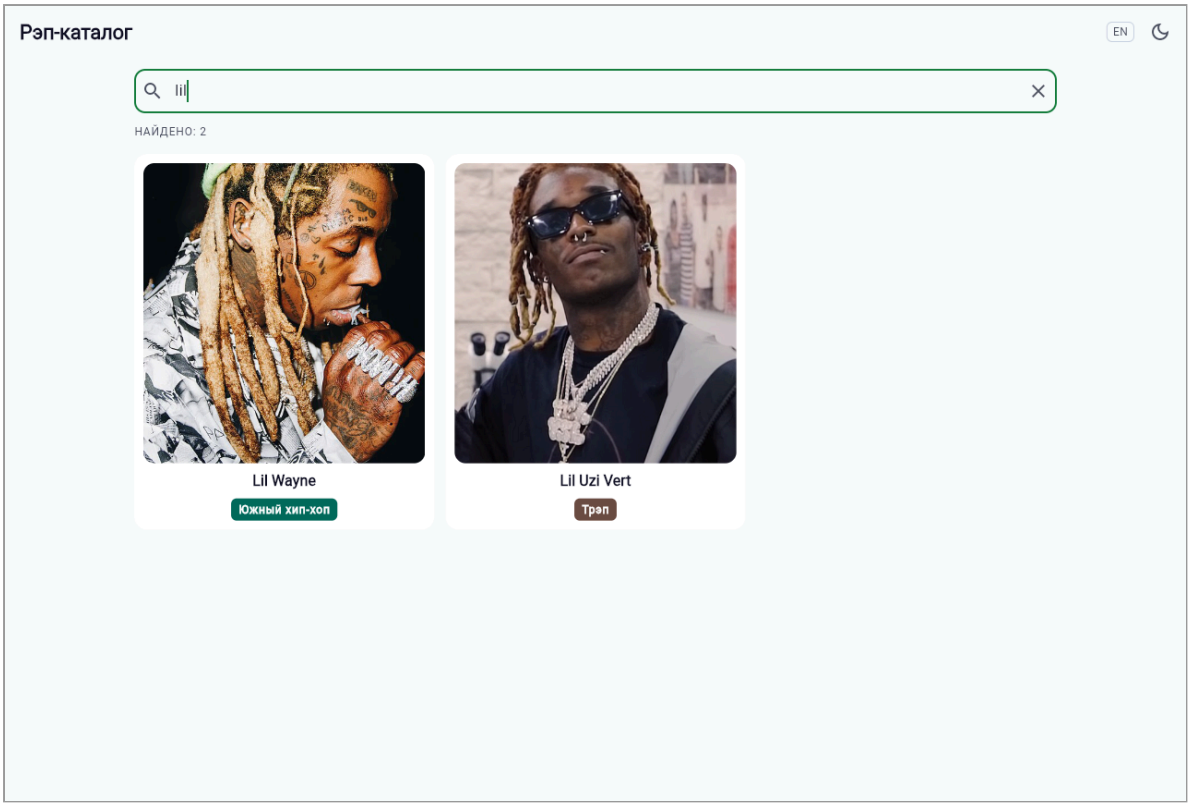


Рисунок 6 — Поиск: подстрока «lil» сужает сетку

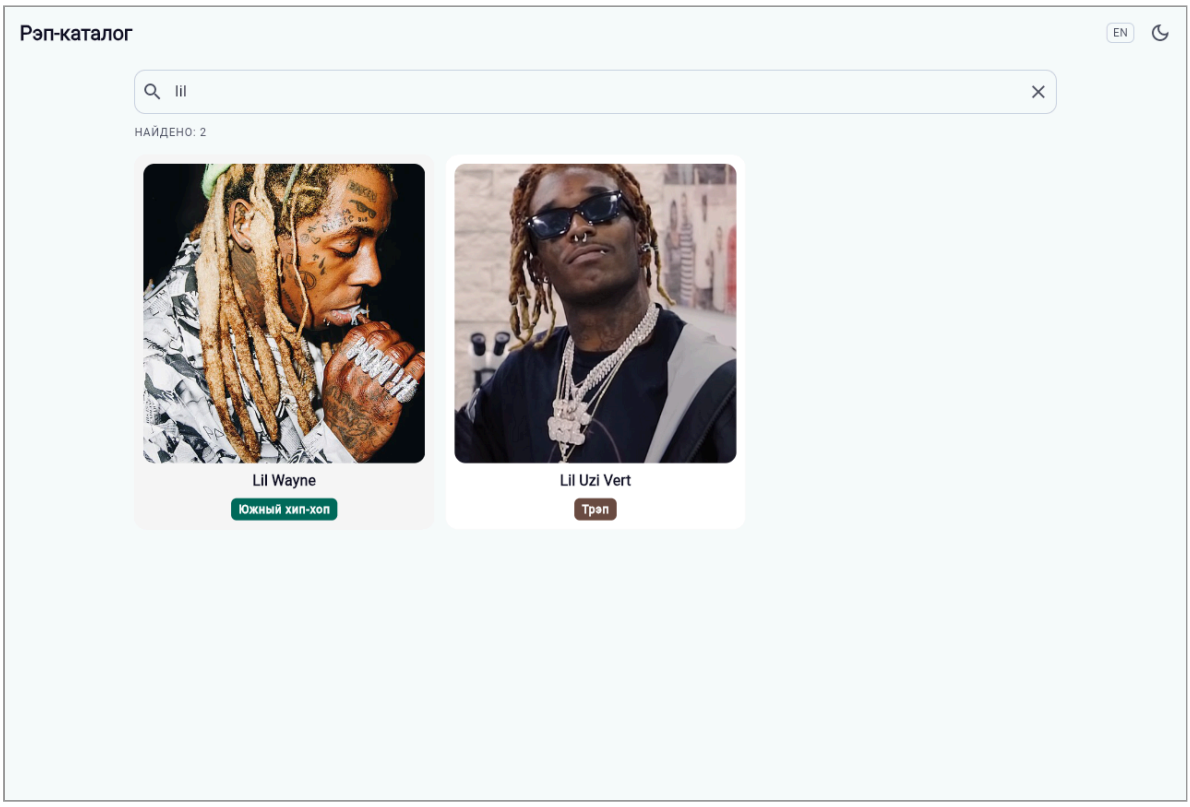


Рисунок 7 — Возврат с детали: запрос сохранён

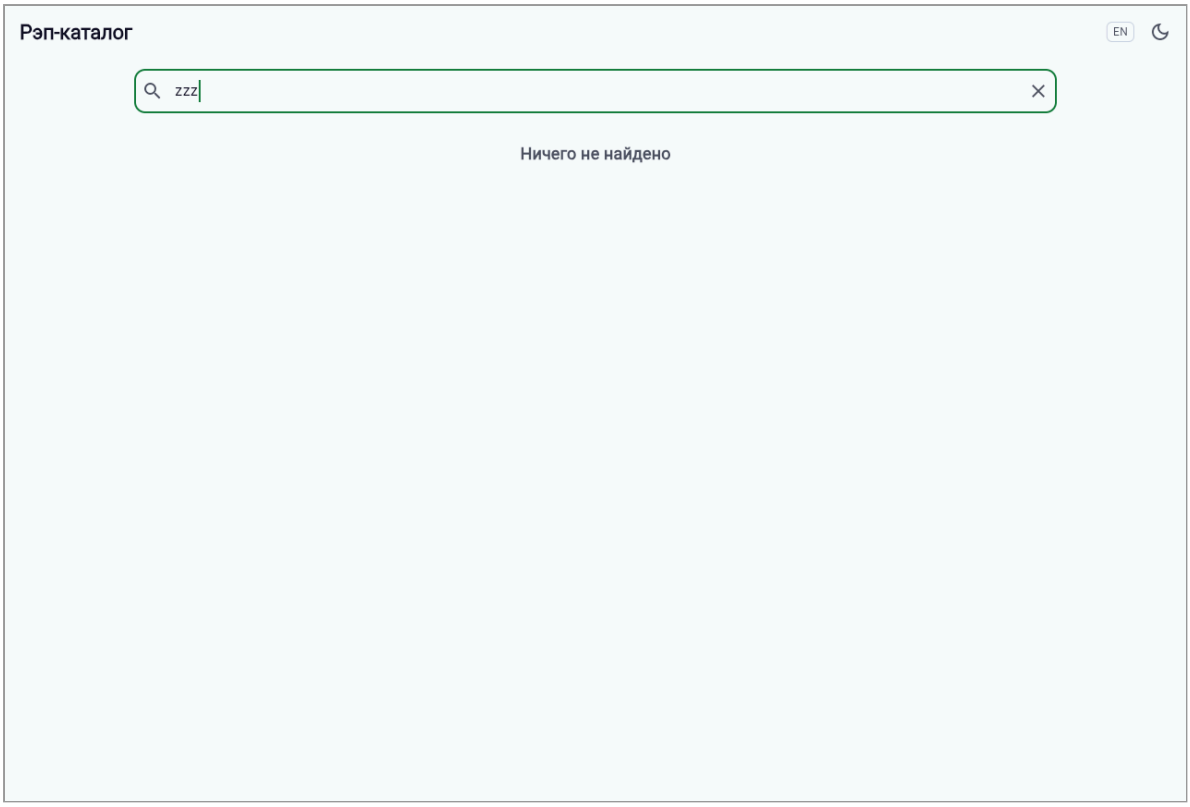


Рисунок 8 — Поиск без совпадений

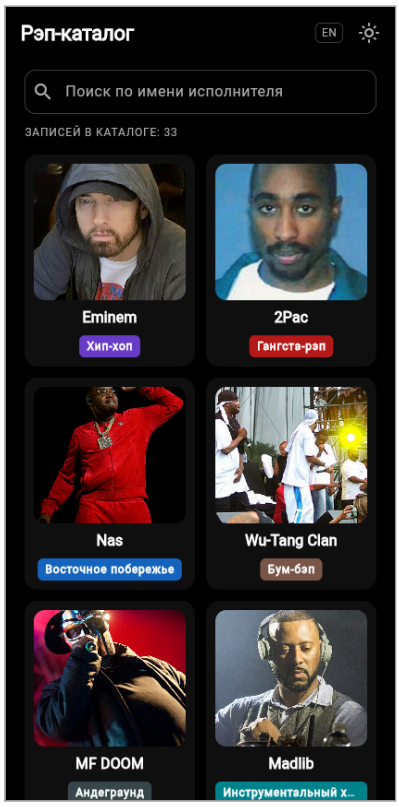


Рисунок 9 — Узкое окно: сетка в две колонки

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы поставленная цель достигнута: каркас приложения «Рэп-каталог» перенесён на Flutter. Все поставленные задачи выполнены.

Код разложен по фичам со слоями data, domain и presentation, репозиторий описан интерфейсом и регистрируется в корне приложения, поэтому замена моков на сеть на следующей вехе затронет только слой данных. Моковый набор из 33 артистов повторяет форму ответов MusicBrainz.

Управление состоянием построено на Cubit с закрытыми неизменяемыми состояниями; выбор обоснован прямым характером действий на экранах. Реализованы список с поиском, деталь с переходами к связанным артистам, навигация go_router с идентификатором в пути, светлая и тёмная темы, интерфейс на двух языках с проверкой совпадения ключей и адаптивная сетка.

Сравнение с первой работой показало, что архитектура переносится между фреймворками почти без изменений: слои, модель, состояния экранов и правила остались прежними, поменялись только механизмы — Cubit вместо ViewModel, go_router вместо Navigation 3, ARB вместо строковых ресурсов Compose. Практическая ценность работы состоит в полученном опыте построения приложения на Flutter и в каркасе, на котором следующая веха добавит работу с сетью, состояния загрузки и ошибки и постраничную загрузку.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Flutter documentation [Электронный ресурс]. — URL: https://docs.flutter.dev/ (дата обращения: 07.11.2026).

2. Bloc library : документация flutter_bloc [Электронный ресурс]. — URL: https://bloclibrary.dev/ (дата обращения: 07.11.2026).

3. go_router : пакет навигации для Flutter [Электронный ресурс]. — URL: https://pub.dev/packages/go_router (дата обращения: 07.11.2026).

4. Internationalizing Flutter apps : документация Flutter [Электронный ресурс]. — URL: https://docs.flutter.dev/ui/accessibility-and-internationalization/internationalization (дата обращения: 07.11.2026).

5. Material Design 3: цветовые схемы и токены [Электронный ресурс]. — URL: https://m3.material.io/styles/color/system (дата обращения: 07.11.2026).

6. MusicBrainz API : описание программного интерфейса [Электронный ресурс]. — URL: https://musicbrainz.org/doc/MusicBrainz_API (дата обращения: 07.11.2026).